INKSY

A from-scratch generative AI handwriting studio

End-to-End Project Plan

Product definition, model design, rendering, evaluation and deployment

Project promise

Build a web application that turns typed text into handwritten-style pages using a generative character model trained by us. No pretrained handwriting model and no external AI generation API are used.

Version 1.1 | Revised 26 October 2026

This revision integrates the gap review into the original blueprint. It clarifies the character model, supported text, visual quality, multi-page export, API behavior, resource limits and delivery gates.

Planning status: Proposed implementation baseline. Numerical targets, hardware assumptions and cost limits require validation; they are not measured results or approved expenditure.

Scope: Isolated-character generation and page composition for the first public release. Natural connected handwriting remains a separate research milestone.

This document updates the plan only. It does not indicate that implementation, training, data collection or deployment has started.

PROJECT BLUEPRINT / 01

Purpose and revision summary

Inksy is a browser-based handwriting generator. A user enters text, configures a page, generates a preview and downloads the result. The central technical deliverable is a reproducible, custom-trained PyTorch model integrated into a working product.

Primary MVP audience and success

Recommended baseline: an educational / portfolio demo for people exploring generated handwriting. It should produce legible, visually coherent pages while explaining its experimental limitations. Print-ready document production and exact transcription guarantees are outside the first release.

Gap addressed	Resolution in this revision
Audience and visual bar	Define the demo audience; freeze a representative passage and a visual reference before training.
Conditioning after training	Use a class-conditional VAE in the first product-model experiment.
Character and punctuation coverage	Select EMNIST ByClass; define an explicit alphabet and disclose procedural punctuation.
Page coherence and glyph geometry	Add consistency checks, character metrics, baseline rules and bounded variation.
Subjective quality gates	Add provisional numerical targets, per-class checks and an untouched final test set.
Multi-page output and API behavior	Specify a response manifest, page previews, PNG/ZIP downloads and seed semantics.
Resource and delivery assumptions	Add request budgets, concurrency controls, compute decisions and a revised schedule.

How to read this plan

Recommended defaults make the plan concrete without pretending experiments have already succeeded. Freeze them during scope definition. Validate quality and speed in the early model gate; record any change before final testing.

The original architecture, training separation, privacy principles, repository practices, deployment checks and later sequence-model roadmap are retained and expanded.

PROJECT BLUEPRINT / 02

Product scope and user flow

Step	User experience	System responsibility
1. Enter text	Type or paste a passage.	Show character support, normalization changes and input limits before generation.
2. Configure	Choose page size, paper, spacing, glyph size and variation.	Validate combinations and estimate page count; use bounded controls.
3. Generate	Start generation and see a loading state.	Submit one request, sample conditioned glyphs and compose bounded pages.
4. Preview	Inspect all pages and any warnings.	Display page navigation, dimensions, seed, model version and timing.
5. Export	Download a page or all pages.	Download the same rendered output shown in the preview.

In the first release

• Generated uppercase/lowercase Latin letters and digits; explicit punctuation and whitespace handling; no silent loss of text.

• Plain, ruled and grid paper; glyph size, character/word spacing, line spacing and margins; bounded random variation.

• Multi-line and multi-page rendering, preview, regenerate, PNG download for one page and ZIP of PNGs for all pages.

• Model-information page, reproducible training instructions, held-out evaluation and a deployed frontend/backend.

Outside the first release

Perfect cursive continuity, copying a person's handwriting, signature generation, handwriting recognition as a core feature, rich-text editing, PDF/vector export, accounts, saved history and batch jobs. Multiple learned styles require evidence that the data and model can support them.

Visual reference and canonical passage

Before training, choose a visual reference for legibility, spacing, slant and ink consistency. Use a 500-1,000-character passage containing mixed case, digits, punctuation, repeated letters, paragraphs and long words. Add a glyph sheet covering every supported symbol. The reference illustrates a quality bar; it is not a target writer to imitate.

Product acceptance

A user can create a legible multi-page result with every supported input character accounted for, inspect its limitations and download exactly what was previewed. Visual realism and model correctness are evaluated separately.

PROJECT BLUEPRINT / 03

Supported text and punctuation

Choose **EMNIST ByClass** for the initial product dataset: its 62 classes provide separate labels for A-Z, a-z and 0-9. Do not substitute the Letters or Balanced configurations without revising case handling and the product contract. ByClass is unbalanced, so inspect and report performance for each class. [1]

Input	Proposed v1 behavior
A-Z, a-z, 0-9	Generate from the conditional model. Preserve the requested class; document any visually ambiguous pairs.
Space and newline	Render as layout instructions, not generated images. Preserve paragraph breaks and repeated spaces within limits.
Tab and line endings	Normalize tabs to four spaces; normalize CRLF/CR to LF. Show the normalized text before generation.
Punctuation	Support . , ! ? : ; apostrophe, double quote, hyphen, and parentheses using procedural Pillow paths.
Smart punctuation	Offer explicit conversion of curly quotes to straight quotes, long dashes to hyphen and ellipsis to three dots. Confirm through the displayed normalized text.
Other symbols	Reject with a structured list of unsupported symbols and positions. Do not silently drop, transliterate or substitute them.
Empty / whitespace-only input	Reject with a helpful validation error. Reject disallowed control characters.

Why punctuation is an explicit exception

EMNIST does not supply the punctuation classes required by this MVP. [1] The recommended baseline draws punctuation procedurally and discloses that in the README, model-information page and evaluation report. All letters and digits still come from the trained generative model; no font replaces them.

Alternative if every visible glyph must be learned

Replace procedural punctuation with a consented, labelled supplementary dataset and retrain the model with those classes. Record provenance, author permissions, coverage and held-out splits. Estimate this separately before committing to the release date.

The model-information endpoint must expose the supported alphabet, normalization rules and punctuation strategy. The frontend and backend must use the same versioned character contract.

PROJECT BLUEPRINT / 04

System architecture and repository

Keep offline training separate from online inference. The API loads a versioned checkpoint once at startup; user requests never trigger model training.

Layer	Technology and responsibility
Frontend	Next.js, React, TypeScript, Tailwind CSS: editor, validation hints, settings, loading/errors, page previews and downloads.
API	FastAPI and Pydantic: authoritative validation, model loading, generation, metadata, bounded concurrency and error responses.
Model	PyTorch conditional convolutional VAE: requested class plus sampled latent vector produce a grayscale glyph.
Composition	Pillow: foreground masks, glyph geometry, spacing, page layout, backgrounds, PNG encoding and ZIP packaging.
Training / evaluation	EMNIST, torchvision, NumPy, TensorBoard, Matplotlib: versioned data pipeline, experiments, held-out tests and reports.
Operations	Container or persistent Python service, structured logging, health/readiness checks and release rollback. No database for the MVP.

Request flow

Browser editor → FastAPI validation → batched conditional glyph generation → geometry and page composition → JSON manifest containing encoded PNG pages → browser preview / PNG / ZIP download.

Repository layout

Path from repository root	Contents
ml/	dataset.py, model.py, train.py, generate.py, evaluate.py, tests/
api/	main.py, schemas.py, services/, tests/
web/	Next.js app/, components/, lib/, browser tests
data/ and artifacts/	Ignored datasets/cache; checkpoints, configurations and experiment outputs. Store release artifacts outside Git.
docs/ and root files	Model card, evaluation, API contract, decision log, runbook; README, LICENSE, dependency lockfiles and .gitignore.

An optional browser canvas may support interactions. The server-rendered page remains the authoritative generated output. Use Git/GitHub with linting, meaningful automated tests and CI.

PROJECT BLUEPRINT / 05

Conditional generative model

Use a **class-conditional convolutional VAE** from the first product training run. The model learns variations within a requested character class. An unconditional VAE can be a small learning exercise, but it is not the product checkpoint.

Component	Initial specification
Input	Upright 28 x 28 grayscale image x, class label y and versioned normalization.
Encoder	Small convolutional network conditioned on y, producing latent mean and log-variance for q(z | x, y).
Latent	Sample z with the reparameterization trick during training. Experiment with a small dimension range, such as 16 / 32 / 64.
Decoder	Decode z plus the requested class y into an image. Use one shared conditional model rather than one model per letter by default.
Objective	Reconstruction loss plus weighted KL divergence to the prior. Record loss reduction/scaling, likelihood choice and KL weight or schedule.
Inference	Sample z from the trained prior and supply y directly. Do not generate arbitrary classes and hope they match the text.
Training	Mini-batches, Adam or equivalent, validation-based checkpoint selection, early stopping and fixed-seed per-class sample grids.

Variation and writing consistency

Class conditioning controls identity, not a shared writer style. A common random seed does not prove style coherence. Begin with bounded latent variation and conservative page-wide slant, ink and baseline controls. Evaluate repeated letters and full sentences for consistent appearance; do not label controls as learned styles without evidence.

Model experiments and limits

Compare a small documented set of latent sizes, reconstruction/KL settings and decoder variants within the compute budget. Inspect prior-generated samples as well as reconstructions. Check for blurry strokes, class confusion, posterior collapse and near-duplicate outputs. Record failed runs and selection reasons.

Early exit gate

Before building the polished UI, demonstrate requested-character generation across the proposed alphabet and a minimally composed paragraph. If quality fails, narrow the supported set or revise the model/data plan and timeline. Do not hide failure behind font rendering.

PROJECT BLUEPRINT / 06

Data, training and reproducibility

Dataset preparation

• Obtain EMNIST from the official source. Record the dataset configuration, source URL, download date, checksum and applicable usage/redistribution conditions. Do not infer permissions from a third-party mirror. [1]

• Inspect every label mapping, image orientation, foreground convention, shape, dtype and normalization. Save labelled sample grids before training; visually verify easily confused symbols.

• Keep the official test partition untouched. Create a deterministic, stratified validation subset from the training partition; use validation only for model selection. Document sample counts per class and any duplicate checks.

• Inspect class imbalance. Choose class-balanced sampling or another documented strategy if required, and evaluate class coverage separately from aggregate metrics.

• Use only label-preserving augmentation. Avoid transforms such as flips or large rotations that can change character identity. Record all transformations.

Experiment record stored with every checkpoint

Record	Required fields
Data identity	Source/checksum, configuration, exact class mapping, train/validation/test membership and preprocessing version.
Training identity	Code revision, dependency versions, hardware, optimizer, learning rate, batch size, latent dimension, loss settings and random seeds.
Results	Training/validation curves, per-class prior samples, reconstructions, timings, selected checkpoint and selection rationale.
Release compatibility	Model architecture/configuration, tensor shapes, supported alphabet, artifact hash and compatible renderer/API versions.

Compute plan before committing to dates

Name the available training machine, CPU/GPU, RAM/VRAM and storage. Set a maximum experiment count, training-hour allowance and spending cap before using paid compute. Time one short run to estimate the full search. No hardware or cloud budget is assumed to have been approved by this document.

Later line-level research may use IAM or another suitable source, subject to access and usage terms. That dataset review and new training effort belong to the v2 research milestone. [2]

PROJECT BLUEPRINT / 07

Glyph geometry and page composition

A page must remain readable after the glyph model succeeds. The renderer needs explicit metrics because centred, isolated training images do not provide a complete typography or baseline system.

Concern	Required behavior
Ink extraction	Convert generated foreground to a clean grayscale/alpha mask; remove padding conservatively without cutting dots or thin strokes.
Character metrics	Maintain versioned advance width, baseline offset, nominal height and ascender/descender category. Calibrate using generated grids.
Proportions	Preserve aspect ratio. Give narrow and wide letters different advances; avoid stretching every character into an equal-size box.
Vertical placement	Align normal letters to the baseline; allow g, j, p, q and y to descend; preserve capitals, dots and tall letters.
Spacing and wrapping	Use word-aware wrapping. Preserve explicit newlines; split a word only when it cannot fit, without silently adding a hyphen.
Page boundaries	Reserve margins before layout. Start a new page on overflow; reject the request if the configured page/pixel limit would be exceeded.
Variation	Bound jitter, rotation and baseline variation to avoid collisions. Apply page-wide ink/slant controls consistently.
Backgrounds	Plain, ruled and grid options; keep ink visible against the selected background. Test alpha compositing for white-box artifacts.

Proposed output defaults

Use an A4-ratio raster page at 1240 x 1754 pixels (approximately 150 dpi at A4 size). Limit all output using the shared pixel budget. Expose a small set of tested glyph sizes; start around the model's native scale and validate any enlargement visually.

Larger canvas dimensions do not add learned stroke detail to a 28 x 28 glyph. Do not describe an upscaled output as high-resolution handwriting. Establish a readable digital-output quality bar before considering print-quality claims.

Geometry regression cases

Test mixed widths (iiiWWW), ascenders/descenders (bdfhgjpqy), punctuation, repeated spaces, blank lines, long unbroken words, page boundaries and extreme valid settings. Verify reading order, no missing glyphs, no clipping and no unintended overlap.

PROJECT BLUEPRINT / 08

Evaluation and release quality gates

Evaluate reconstruction, prior generation and composed pages separately. The targets below are **provisional acceptance targets**, not achieved scores. Freeze the final values after baseline validation and before opening the final test set; record the rationale for any change.

Dimension	Protocol and proposed gate
Class correctness	Generate at least 100 fixed-seed samples per alphanumeric class (6,200 total). Target at least 99% aggregate requested-class accuracy and 97% in every class; report macro accuracy and confusion matrix.
Evaluator quality	If using a separate classifier, train it from scratch and measure it on real held-out characters. Human-review at least 20 generated samples/class plus disagreements; report ambiguous samples and evaluator limitations.
Page readability	Review at least 10 passages of 500-1,000 characters across multiple seeds, including the canonical passage. Target at most 1% incorrect/unreadable alphanumeric glyphs; report exact counts and examples.
Geometry / text coverage	All deterministic fixtures pass: supported characters preserved in order, correct wrapping/page count, no clipping and no unintended collisions at tested settings.
Diversity / coherence	Publish repeated-character grids and latent traversals. Review at least five full pages for coherent slant, thickness and scale; identify collapsed/repeated outputs. Freeze a review rubric in Phase 0.
Reproducibility	Same normalized input, settings, seed and version tuple gives identical pixels on the pinned deterministic CPU runtime. State limitations for other runtimes.
Robustness / speed	Validation, overload and concurrency checks pass. Benchmark latency and peak process memory against the service targets on page 12.

Interpret the numbers honestly

At 99% character correctness, a 1,000-character output can still contain about ten errors. This is an experimental-demo threshold, not a guarantee of faithful document transcription. Reviewers must see representative failures, not only selected attractive samples.

Keep the final test set untouched until architecture and hyperparameters are fixed. For final reporting, include sample counts, seeds, reviewer protocol, class coverage, timing hardware and known limitations. A failed gate requires a documented scope/model decision before release.

PROJECT BLUEPRINT / 09

API contract and error behavior

Use a synchronous, bounded MVP API. Return a JSON manifest with PNG pages encoded as base64 so multi-page metadata and images have one defined response shape. This avoids persistent user-content storage; account for the encoding overhead in response limits.

Endpoint	Contract
GET /health	Liveness only. No inference and no user content.
GET /ready	Readiness: checkpoint loaded, compatible configuration and renderer available. Return unavailable until ready.
GET /model-info	Model/hash, renderer and preprocessing versions, alphabet, normalization/punctuation policy, limits and known limitations.
POST /generate	Accept text, optional seed and typed layout options. Return a manifest with pages and metadata; use no-store caching headers.

Request fields

text; optional integer **seed**; **options** containing page preset, glyph size, margins, character/word/line spacing, background and bounded variation. The backend validates the full combination against the versioned schema; it is authoritative even if the UI has already checked it.

Success response

request_id, normalized_text, normalization_changes, seed, model_version, model_hash, renderer_version, config_version, generation_ms, page_count, warnings, and **pages[]**. Each page contains an index, width_px, height_px, media_type and png_base64. Do not log the text or payload. No temporary public image URLs are needed for this baseline.

Error response

Use a consistent envelope: **request_id, code, message, details**. Return 422 for unsupported characters/invalid options, 413 for body/output limits, 429 for rate or capacity limits, 503 when the model is unavailable, and a sanitized 500 for unexpected failures. Include unsupported symbols and positions without leaking server internals.

Contract checks

Version the schema; test frontend/backend compatibility, deterministic output, output limits, error cases and a real-checkpoint integration request. Seed omission must generate and return a seed. Never retry a costly request automatically without a bounded, explicit policy.

PROJECT BLUEPRINT / 10

Preview, export and seed semantics

User interface states

State / action	Required behavior
Before generation	Show supported text and normalization changes, tested controls and limits. Prevent empty submissions; preserve the editor when validation fails.
Generating	Use an indeterminate loading state with elapsed time. Disable duplicate submissions. Do not show invented percentage progress for a synchronous request.
Successful preview	Display every page with previous/next navigation, page count, dimensions, seed, version and warnings. Mark a preview as outdated when text/settings change.
Regenerate	Create a new seed and render again. Provide a separate way to reuse a seed for reproducibility.
Download page	Decode the returned PNG and download that same artifact; do not run generation again.
Download all	Build a ZIP from the returned PNG bytes with ordered, safe filenames such as inksy-page-001.png.
Failure / overload	Keep text and settings, explain the actionable error and offer a deliberate retry. Do not expose tracebacks or silently truncate text.

Reproducibility contract

A seed identifies random choices only within a specified model and rendering configuration. Store the effective seed in browser memory with the normalized text, options and version tuple. Use a request-local random generator; concurrent requests must not modify a shared global RNG.

Target deterministic pixel output on the pinned CPU environment. A model update, preprocessing change, different renderer, page setting or device/runtime change can alter the result. Document the supported reproducibility boundary in model information.

Accessibility and responsiveness

Provide labelled inputs, keyboard-operable controls, visible focus, accessible loading/error announcements, sufficient contrast and useful preview descriptions. Keep the original text available beside the image. Test the editor, page navigation and download actions on small and large screens.

Accounts and saved history remain deferred. Editor data and generated images are held in memory for the active session; document any later persistence or analytics change before introducing it.

PROJECT BLUEPRINT / 11

Performance, limits and security

Use these proposed engineering budgets to size the first benchmark. Final deployment limits must be based on measured behavior. Enforce limits on the backend, including invalid combinations and total output across all pages.

Budget	Proposed initial value / policy
Text and body	Maximum 3,000 Unicode code points after normalization and a 32 KiB request body; also bound raw text before normalization.
Pages and dimensions	Maximum three pages, maximum 2048 pixels per side, and maximum seven million pixels across the request.
Encoded response	Maximum 12 MiB for the JSON response, including base64 overhead; fail clearly before returning an oversized response.
Concurrency	Start with one inference worker/process and one active generation. Reject excess work with 429; no unbounded in-memory queue.
Public request rate	Initial limit of five generation requests/minute per source IP with a small burst allowance; adjust for measured demand and shared-network users.
Latency target	On a declared 2-vCPU / 4-GiB CPU benchmark host: warm p95 at most 5 seconds for a 500-character one-page request; at most 15 seconds at the supported maximum.
Memory / timeout	Target peak process RSS below 2.5 GiB with the active request. Use a 20-second generation deadline with bounded stages; align upstream timeouts.
Training budget	Hardware, maximum experiment count, training hours and paid-spend cap must be entered in the decision register before training.

Implementation and measurement

Batch glyph inference; profile model sampling, layout and encoding separately. Bound batch size and image allocation before allocating buffers. Benchmark at least 30 warm requests for each representative size and report sample counts, cold-start timing, peak memory and overload behavior. Keep admission control and model work off paths that would stall health checks.

Privacy and abuse controls

Use HTTPS, strict allowed CORS origins, input schemas and safe filenames. CORS does not prevent direct API calls: enforce rate/admission limits independently. Log request IDs, lengths, timings and error categories; exclude full text and images, including from error-monitoring tools. Use no-store responses and no default persistence. Give the demo a clear acceptable-use statement; do not market impersonation, signature forgery or false official documents.

If the baseline fails speed or memory targets, reduce limits or revise serving architecture before launch. Multiple workers each load their own checkpoint and must be budgeted accordingly.

PROJECT BLUEPRINT / 12

Build phases and delivery gates

The revised sequence resolves conditioning before product training and tests the visual premise before frontend polish. Estimates are working days for one focused developer, assuming suitable compute and no new punctuation dataset.

Phase	Work and exit gate	Days
0. Scope and setup	Freeze audience, alphabet, punctuation exception, reference passage, metrics, limits, hardware and budget; establish repo/environment and decision log.	2-3
1. Data pipeline	Verify labels/orientation and provenance; create deterministic splits, per-class counts and labelled sanity grids.	3-5
2. Conditional model	Implement/train cVAE, evaluate requested classes and prior samples, compare bounded experiments and compose a debug paragraph. Pass the early visual/identity gate.	6-10
3. Composition	Implement glyph metrics, spacing, backgrounds, wrapping, page limits and PNG output. Pass geometry fixtures and paragraph review.	4-6
4. API	Add checkpoint loading, schemas, manifest response, limits, seed isolation, errors, health/readiness and integration tests.	3-5
5. Frontend	Build editor, controls, page navigation, loading/errors, regenerate, PNG/ZIP downloads and accessibility.	4-7
6. Integration / QA	Run frozen quality protocol, final held-out evaluation, concurrency/limit benchmarks, browser tests and documentation review.	4-6
7. Deployment	Containerize, select measured instance size, configure HTTPS/CORS/logging, smoke-test and verify rollback.	2-4

Revised schedule

Base estimate: **28-46 working days**, approximately **6-10 five-day weeks**. With a 20-30% contingency, plan approximately **7-12 weeks**. Extra dataset collection, major model changes and natural cursive research require separate estimates.

For comparison, the original phase ranges totalled 24-45 working days, which was wider than its stated 4-7-week summary. These remain planning ranges, not a delivery commitment. Stop/replan gates occur after data sanity checks, model feasibility, final QA and production smoke tests.

PROJECT BLUEPRINT / 13

Deployment and operations

Piece	Recommended approach
Frontend	Vercel or another Next.js-compatible host. Set the production API URL; keep secrets out of client variables.
API and model	Persistent Python service or container with measured CPU/RAM headroom. Start with CPU inference; introduce GPU serving only if benchmarks justify it.
Checkpoint	Bundle a versioned artifact or fetch from controlled storage at startup. Verify its hash and trusted origin; never reload per request.
Generated files	No server-side persistence for the baseline. Return bounded page payloads and create downloads in the browser.
CI / release	GitHub lint/test workflow, known commit/tag, pinned dependencies, trusted artifact and a documented rollback release.

Release procedure

• Prepare the backend image with pinned Python dependencies and a non-root runtime where practical. Verify model/configuration compatibility at startup.

• Choose an instance from measured latency, peak RSS, startup time and worker count. Record monthly hosting/artifact-storage estimate and any free-tier sleep or timeout constraints before purchase.

• Deploy and verify /health, /ready, /model-info and a real /generate request over HTTPS. Keep the service unready if the checkpoint is missing or incompatible.

• Deploy the frontend, configure its API origin and restrict CORS. Check actual response limits across any reverse proxy or frontend hosting path.

• Smoke-test mixed case, punctuation, unsupported text, long input, multi-page navigation, regenerate, PNG/ZIP download and overload behavior.

• Verify logs contain no full text/images. Exercise restart and rollback, then record the deployed model, renderer, config and application revisions.

Monitoring and recovery

Track readiness failures, error rates, generation p50/p95, overload rejections, process memory and restarts. Set practical alert thresholds after the first benchmark. Keep a known-good release and artifact available. Roll back on failed readiness, broken generation or an unacceptable error/latency regression.

Deployment definition of done

The public UI reaches the production API, the real trained model is used, all pages/downloads work, health and logs are operational, rollback is documented, and a new developer can reproduce the local setup from the README.

PROJECT BLUEPRINT / 14

Engineering practices and deliverables

Reproducible development

• Separate Python and Node.js environments; pin and lock dependencies. Document exact commands for the final repository layout rather than relying on illustrative snippets.

• Keep datasets, model weights, secrets and user-generated content out of Git. Use controlled artifact storage for checkpoint releases and an explicit software license for original code.

• Store training config, split identity, seeds, source revision and evaluation results beside each checkpoint. Record model/renderer compatibility and artifact checksums.

• Keep the API stateless. Avoid a database until accounts/history justify persistence and an explicit retention policy.

Meaningful automated checks

Area	Checks
ML / data	Tensor shapes, finite loss, reparameterization, label mapping, preprocessing orientation and conditional-label propagation.
Renderer	Text coverage/order, proportional widths, baseline/descender placement, whitespace, wrapping, page limits and output dimensions.
API	Schemas, unsupported symbols, response limits, manifest shape, readiness, overload, sanitized errors and seed isolation under concurrency.
End to end	Generate with the release checkpoint, navigate pages, regenerate, preserve preview/download pixels and extract a ZIP with the expected PNG pages.
CI boundaries	Use a small trusted smoke checkpoint or explicit fixture for quick CI. Keep release-quality tests with the real trained checkpoint; do not treat mock output as model evidence.

Required project documentation

README: purpose, exact setup/run commands, limits, punctuation exception and deployment URL. **Model card:** architecture, data provenance, from-scratch constraint, training hardware/configuration, evaluation protocol/results and failures. **Runbook:** deployment, model artifact handling, configuration, monitoring and rollback. **Decision log:** accepted defaults and reasons for scope or threshold changes.

Before choosing data or publishing artifacts, review original usage conditions and document attribution/redistribution obligations. This plan does not establish commercial-use clearance for a dataset or a future release.

PROJECT BLUEPRINT / 15

Risks and version roadmap

Risk	Mitigation / decision trigger
Blurry or wrong glyphs	Tune the conditional model and loss within budget; report per-class failures. Narrow the supported set or replan if the identity/visual gate fails.
Inconsistent-looking page	Evaluate full passages and repeated letters. Limit variation and improve metrics; do not promise learned writing styles without supporting data.
Weak punctuation match	Disclose procedural punctuation, calibrate size/placement and review mixed text. A fully learned alternative requires new labelled data.
Disconnected handwriting	State the isolated-character limitation clearly. Connected cursive needs new data/model research, not just compositor adjustments.
Slow / oversized requests	Batch inference, cap total pixels/payloads, enforce admission limits and benchmark maximum requests. Reduce limits before launch if needed.
Evaluator overconfidence	Validate the classifier on real test data; use human review, per-class results and passage-level evidence.
Schedule / compute overrun	Time a pilot run, cap experiments/spend and use explicit gates. Re-estimate new datasets or architectural changes.
Data or privacy obligations	Record provenance and terms; exclude user content from logs/storage. Revisit policy before persistence or new dataset use.

Version roadmap

Version	Focus and outcome
v0.1 - ML proof	Reproducible conditional character generation, per-class evaluation and an early composed-paragraph review.
v0.2 - Local product	Renderer, bounded API, local browser flow and working PNG/ZIP export with reproducible seeds.
v1.0 - Public MVP	Quality gates, measured deployment, accessibility, documentation and an honest experimental demo.
v1.5 - Better glyphs	Improve sharpness, class accuracy, page consistency and validated variation controls.
v2.0 - Natural writing	Research sequence/stroke or line-level synthesis on suitable data with reviewed terms. Budget separately.

PROJECT BLUEPRINT / 16

Launch checklist and open decisions

Before implementation

• Freeze primary audience, visual reference, canonical passage and exact character support; accept or replace the procedural punctuation exception.

• Record hardware, experiment/training-hour allowance, spending cap and revised delivery range. Freeze the evaluation protocol and initial API/resource budgets.

Before the public release

• Verify official data provenance, mapping, orientation and untouched final test data. Save the conditional model, configuration, hashes, curves and sample grids.

• Pass requested-class, readability, coherence and geometry gates; publish real results and representative failures. Document every failed or adjusted target.

• Pass API validation, rate/admission limits, seed isolation, output-size, long-input and concurrency checks. Record latency, memory and cold-start measurements.

• Verify preview fidelity, page navigation, regenerate, safe PNG/ZIP exports, keyboard access and responsive layouts using the real model.

• Complete HTTPS/CORS/readiness/logging checks, confirm no user-content logging, verify rollback and publish setup guide/model card/runbook.

Decision still to record	Owner / timing
Visual reference and acceptable demo appearance	Project owner; freeze in Phase 0. No reference artifact has been selected yet.
Punctuation exception or supplementary dataset	Project owner; settle before model/data implementation.
Training hardware and maximum paid spend	Project owner; record before training or any paid resource use.
Final metric thresholds and service limits	Developer/project owner; freeze after baseline validation, before final testing.
Hosting provider, instance size and recurring budget	Project owner; choose after benchmarks, before deployment.

Sources and revision basis

Original document: Inksy_End_to_End_Project_Plan.pdf, version 1.0 (October 2026), supplied by the project owner. Version 1.1 integrates the nine-point gap review; no training, performance or implementation results are claimed.

[1] NIST, The EMNIST Dataset. Official configuration and dataset description. Verified 26 October 2026. nist.gov/itl/products-and-services/emnist-dataset

[2] IAM Handwriting Database. Official dataset site for later research; access/usage review remains a future prerequisite. fki.tic.heia-fr.ch/databases/iam-handwriting-database